

Operator Overloading

修改自林仲志與書商投影片
長庚大學人工智慧學系

Learning Objectives

- Basic Operator Overloading
 - Unary operators
 - As member functions
- Friends and Automatic Type Conversion
 - Friend functions, friend classes
 - Constructors for automatic type conversion
- References and More Overloading
 - << and >>
 - Operators: = , [], ++, --

Operator Overloading Introduction

- Operators +, -, %, ==, etc.
 - Really just functions!
- Simply "called" with different syntax:
 $x + 7$
 - "+" is binary operator with x & 7 as operands
 - We "like" this notation as humans
- Think of it as:
 $+(x, 7)$
 - "+" is the function name
 - x, 7 are the arguments
 - Function "+" returns "sum" of it's arguments

Operator Overloading Perspective

- Built-in operators
 - e.g., +, -, =, %, ==, /, *
 - Already work for C++ built-in types
 - In standard "binary" notation
- We can overload them!
 - To work with OUR types!
 - To add "Chair types", or "Money types"
 - As appropriate for our needs
 - In "notation" we're comfortable with
- Always overload with similar "actions"!

Overloading Basics

- Overloading operators
 - VERY similar to overloading functions
 - Operator itself is "name" of function

- Example Declaration:

```
const Money operator +(      const Money& amount1,  
                             const Money& amount2);
```

- Overloads + for operands of type Money
- Uses constant reference parameters for efficiency
- Returned value is type Money
 - Allows addition of "Money" objects

Overloaded "+"

- Given previous example:
 - Note: overloaded "+" NOT member function
 - Definition is "more involved" than simple "add"
 - Requires issues of money type addition
 - Must handle negative/positive values
- Operator overload definitions generally very simple
 - Just perform "addition" particular to "your" type

```
class Money
{
public:
    Money( );
    Money(double amount);
    Money(int theDollars, int theCents);
    Money(int theDollars);
    double getAmount( ) const;
    int getDollars( ) const;
    int getCents( ) const;
    void input( ); //Reads the dollar sign as well as the amount number.
    void output( ) const;
private:
    int dollars; //A negative amount is represented as negative dollars and
    int cents; //negative cents. Negative $4.50 is represented as -4 and -50

    int dollarsPart(double amount) const;
    int centsPart(double amount) const;
    int round(double number) const;
};

const Money operator +(const Money& amount1, const Money& amount2);
const Money operator -(const Money& amount1, const Money& amount2);
bool operator ==(const Money& amount1, const Money& amount2);
const Money operator -(const Money& amount);
```

Money "+" Definition:

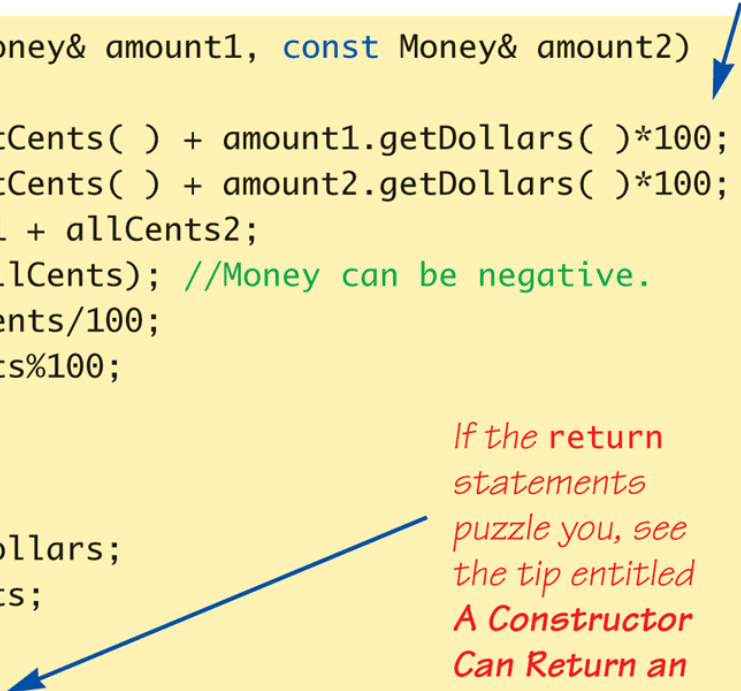
Display 8.1 Operator Overloading

- Definition of "+" operator for Money class:

```
52  const Money operator +(const Money& amount1, const Money& amount2)
53  {
54      int allCents1 = amount1.getCents( ) + amount1.getDollars( )*100;
55      int allCents2 = amount2.getCents( ) + amount2.getDollars( )*100;
56      int sumAllCents = allCents1 + allCents2;
57      int absAllCents = abs(sumAllCents); //Money can be negative.
58      int finalDollars = absAllCents/100;
59      int finalCents = absAllCents%100;

60      if (sumAllCents < 0)
61      {
62          finalDollars = -finalDollars;
63          finalCents = -finalCents;
64      }

65      return Money(finalDollars, finalCents);
66  }
```



*If the return statements puzzle you, see the tip entitled **A Constructor Can Return an Object.***

Definition of “-” operator for Money class:

- `const Money operator -(const Money& amount1, const Money& amount2)`
- `{`
- `int allCents1 = amount1.getCents( ) + amount1.getDollars( )*100;`
- `int allCents2 = amount2.getCents( ) + amount2.getDollars( )*100;`
- `int diffAllCents = allCents1 - allCents2;`
- `int absAllCents = abs(diffAllCents);`
- `int finalDollars = absAllCents/100;`
- `int finalCents = absAllCents%100;`
- `if (diffAllCents < 0)`
- `{`
- `finalDollars = -finalDollars;`
- `finalCents = -finalCents;`
- `}`
- `return Money(finalDollars, finalCents);`
- `}`

Overloaded "=="

- Equality operator, ==
 - Enables comparison of Money objects
 - Declaration:
`bool operator ==(const Money& amount1,
const Money& amount2);`
 - Returns bool type for true/false equality
 - Again, it's a non-member function
(like "+" overload)

Overloaded "==" for Money:

Display 8.1 Operator Overloading

- Definition of "==" operator for Money class:

```
83  bool operator ==(const Money& amount1, const Money& amount2)
84  {
85      return ((amount1.getDollars( ) == amount2.getDollars( ))
86              && (amount1.getCents( ) == amount2.getCents( )));
87  }
```


Constructors Returning Objects

- Constructor a "void" function?
 - We "think" that way, but no
 - A "special" function
 - With special properties
 - CAN return a value!
- Recall return statement in "+" overload for Money type:
 - `return Money(finalDollars, finalCents);`
 - Returns an "invocation" of Money class!
 - So constructor actually "returns" an object!
 - Called an "anonymous object"

Returning by const Value

- Consider "+" operator overload again:
const Money operator +(const Money& amount1,
const Money& amount2);
 - Returns a "constant object"?
 - Why?
- Consider impact of returning "non-const" object to see...→

Overloading Unary Operators

- C++ has unary operators:
 - Defined as taking one operand
 - e.g., - (negation)
 - `x = -y; // Sets x equal to negative of y`
 - Other unary operators:
 - `++`, `--`
- Unary operators can also be overloaded

Overload "-" for Money

- Overloaded "-" function declaration
 - Placed outside class definition:
const Money operator –(const Money& amount);
 - Notice: only one argument
 - Since only 1 operand (unary)
- "-" operator is overloaded twice!
 - For two operands/arguments (binary)
 - For one operand/argument (unary)
 - Definitions must exist for both

Overloaded "-" Definition

- Overloaded "-" function definition:

```
const Money operator –(const Money& amount)
{
    return Money(-amount.getDollars(),
                 -amount.getCents());
}
```
- Applies "-" unary operator to built-in type
 - Operation is "known" for built-in types
- Returns anonymous object again

Overloaded "-" Usage

- Consider:

```
Money    amount1(10),  
          amount2(6),  
          amount3;
```

```
amount3 = amount1 - amount2;
```

- Calls binary "-" overload

```
amount3.output();    //Displays $4.00
```

```
amount3 = -amount1;
```

- Calls unary "-" overload

```
amount3.output()     //Displays -$10.00
```


Overloading as Member Functions

- Previous examples: standalone functions
 - Defined outside a class
- Can overload as "member operator"
 - Considered "member function" like others
- When operator is member function:
 - Only ONE parameter, not two!
 - Calling object serves as 1st parameter

Member Operator in Action

- Money cost(1, 50), tax(0, 15), total;
total = cost + tax;
 - If "+" overloaded as member operator:
 - Variable/object cost is calling object
 - Object tax is single argument
 - Think of as: total = cost.+(tax);
- Declaration of "+" in class definition:
 - const Money operator +(const Money& amount);
 - Notice only ONE argument

const Functions

- When to make function const?
 - Constant functions not allowed to alter class member data
 - Constant objects can ONLY call constant member functions
- Good style dictates:
 - Any member function that will NOT modify data should be made const
- Use keyword *const* after function declaration and heading

Overloading Operators: Which Method?

- Object-Oriented-Programming
 - Principles suggest member operators
 - Many agree, to maintain "spirit" of OOP
- Member operators more efficient
 - No need to call accessor & mutator functions
- At least one significant disadvantage
 - (Later in chapter...)

Overloading Function Application ()

- Function call operator, ()
 - Must be overloaded as member function
 - Allows use of class object like a function
 - Can overload for all possible numbers of arguments
- Example:
Aclass anObject;
anObject(42);
 - If () overloaded → calls overload
 - `const Aclass operator()(const int x);`

Other Overloads

- `&&`, `||`, and comma operator
 - Predefined versions work for bool types
 - Recall: use "short-circuit evaluation"
 - When overloaded no longer uses short-circuit
 - Uses "complete evaluation" instead
 - Contrary to expectations
- Generally should not overload these operators

Friend Functions

- Nonmember functions
 - Recall: operator overloads as nonmembers
 - They access data through accessor and mutator functions
 - Very inefficient (overhead of calls)
- Friends can directly access private class data
 - No overhead, more efficient
- So: best to make nonmember operator overloads friends!

Friend Functions

- Friend function of a class
 - Not a member function
 - Has direct access to private members
 - Just as member functions do
- Use keyword *friend* in front of function declaration
 - Specified IN class definition
 - But they're NOT member functions!


```
class MyClass {
private:
    int myVar;

public:
    MyClass(int var) : myVar(var) {}

    // 宣告 friend 函數
    friend void printVar(MyClass& obj);
};

// friend 函數的定義
void printVar(MyClass& obj) {
    // 直接訪問私有成員 myVar
    cout << "myVar: " << obj.myVar << endl;
}

int main() {
    MyClass obj(10);
    // 使用 friend 函數使用私有成員
    printVar(obj);

    return 0;
}
```

Friend Function Uses

- Operator Overloads
 - Most common use of friends
 - Improves efficiency
 - Avoids need to call accessor/mutator member functions
 - Operator must have access anyway
 - Might as well give full access as friend
- Friends can be any function

Original operator overloading

```
1 #include <iostream>
2 using namespace std;
3
4 class example
5 {
6     public:
7         example(int initData):data(initData){};
8         example():data(0){};
9         int getdata() const;
10        void setdata(int newData);
11    private:
12        int data;
13 };
14 const example operator +(const example &v1, const example &v2);
15
16 int example::getdata()const
17 {return data;}
18
19 void example::setdata(int newData)
20 {data=newData;}
21
22 const example operator +(const example &v1, const example &v2)
23 {
24     return example( v1.getdata()+v2.getdata());
25 }
```

Friend operator overloading

```
1 #include <iostream>
2 using namespace std;
3
4 class example
5 {
6     public:
7         friend const example operator +(const example &v1, const example &v2);
8         example(int initData):data(initData){};
9         example():data(0){};
10        int getdata() const;
11        void setdata(int newData);
12    private:
13        int data;
14 };
15
16 int example::getdata()const
17 {return data;}
18
19 void example::setdata(int newData)
20 {data=newData;}
21
22 const example operator +(const example &v1, const example &v2)
23 {
24     return example( v1.data+v2.data);
25 }
26
```


Friend Function Purity

- Friends not pure?
 - "Spirit" of OOP dictates all operators and functions be member functions
 - Many believe friends violate basic OOP principles
- Advantageous?
 - For operators: very!
 - Allows automatic type conversion
 - Still encapsulates: friend is in class definition
 - Improves efficiency

Friend Classes

- Entire classes can be friends
 - Similar to function being friend to class
 - Example:
class F is friend of class C
 - All class F member functions are friends of C
 - NOT reciprocated
 - Friendship granted, not taken
- Syntax: friend class F
 - Goes inside class definition of "authorizing" class


```
1 #include <iostream>
2 using namespace std;
3
4 class A
5 {
6     friend class B;
7     public:
8     private:
9         int data;
10 };
11
12 class B
13 {
14     public:
15         int handle( A v1, A v2);
16 };
17
18 int B::handle( A v1, A v2)
19 { return v1.data+v2.data};
```

References

- Reference defined:
 - Name of a storage location
 - Similar to "pointer", a kind of alias name
- Example of stand alone reference:
 - `int robert;`
`int& bob = robert;`
 - *bob* is reference to storage location for *robert*
 - Changes made to *bob* will affect *robert*
- Confusing?

References Usage

- Seemingly dangerous
- Useful in several cases:
- Call-by-reference
 - Often used to implement this mechanism
- Returning a reference
 - Allows operator overload implementations to be written more naturally
 - Think of as returning an "alias" to a variable

Returning Reference

- Syntax:
`double& sampleFunction(double& variable);`
 - `double&` and `double` are different
 - Must match in function declaration and heading
- Returned item must "have" a reference
 - Like a variable of that type
 - Cannot be expression like `"x+5"`
 - Has no place in memory to "refer to"

Returning Reference in Definition

- Example function definition:
`double& sampleFunction(double& variable)`
`{`
 `return variable;`
`}`
- Trivial, useless example
- Shows concept only
- Major use:
 - Certain overloaded operators

Overloading >> and <<

- Enables input and output of our objects
 - Similar to other operator overloads
 - New subtleties
- Improves readability
 - Like all operator overloads do
 - Enables:
`cout << myObject;`
`cin >> myObject;`
 - Instead of need for:
`myObject.output(); ...`

Overloading >>

- Insertion operator, <<
 - Used with cout
 - A binary operator
- Example:
`cout << "Hello";`
 - Operator is <<
 - 1st operand is predefined object *cout*
 - From library *iostream*
 - 2nd operand is literal string "Hello"

Overloading >>

- Operands of >>
 - Cout object, of class type ostream
 - Our class type
- Recall Money class
 - Used member function output()
 - Nicer if we can use >> operator:
Money amount(100);
cout << "I have " << amount << endl;
instead of:
cout << "I have ";
amount.output()

Overloaded << Return Value

- Money amount(100);
cout << amount;
 - << should return some value
 - To allow cascades:
cout << "I have " << amount;
(cout << "I have ") << amount;
cout << amount;
 - Two are equivalent
- What to return?
 - cout object!
 - Returns it's first argument type, ostream

Overloaded >> Example:

Display

Display 8.5 Overloading << and >>

```
1  #include <iostream>
2  #include <cstdlib>
3  #include <cmath>
4  using namespace std;

5  //Class for amounts of money in U.S. currency
6  class Money
7  {
8  public:
9      Money( );
10     Money(double amount);
11     Money(int theDollars, int theCents);
12     Money(int theDollars);
13     double getAmount( ) const;
14     int getDollars( ) const;
15     int getCents( ) const;
16     friend const Money operator +(const Money& amount1, const Money& amount2)
17     friend const Money operator -(const Money& amount1, const Money& amount2)
18     friend bool operator ==(const Money& amount1, const Money& amount2);
19     friend const Money operator -(const Money& amount);
20     friend ostream& operator <<(ostream& outputStream, const Money& amount);
21     friend istream& operator >>(istream& inputStream, Money& amount);
22 private:
23     int dollars; //A negative amount is represented as negative dollars and
24     int cents; //negative cents. Negative $4.50 is represented as -4 and -50.
```


Overloaded >> Example:

Display

```
25     int dollarsPart(double amount) const;
26     int centsPart(double amount) const;
27     int round(double number) const;
28 };

29 int main( )
30 {
31     Money yourAmount, myAmount(10, 9);
32     cout << "Enter an amount of money: ";
33     cin >> yourAmount;
34     cout << "Your amount is " << yourAmount << endl;
35     cout << "My amount is " << myAmount << endl;
36
37     if (yourAmount == myAmount)
38         cout << "We have the same amounts.\n";
39     else
40         cout << "One of us is richer.\n";

41     Money ourAmount = yourAmount + myAmount;
```

Overloaded >> Example:

Display

Display 8.5 Overloading << and >>

```
42     cout << yourAmount << " + " << myAmount
43         << " equals " << ourAmount << endl;

44     Money diffAmount = yourAmount - myAmount;
45     cout << yourAmount << " - " << myAmount
46         << " equals " << diffAmount << endl;

47     return 0;
48 }
```

Since << returns a reference, you can chain << like this. You can chain >> in a similar way.

<Definitions of other member functions are as in Display 8.1. Definitions of other overloaded operators are as in Display 8.3.>

```
49 ostream& operator <<(ostream& outputStream, const Money& amount)
50 {
51     int absDollars = abs(amount.dollars);
52     int absCents = abs(amount.cents);
53     if (amount.dollars < 0 || amount.cents < 0)
54         //accounts for dollars == 0 or cents == 0
55         outputStream << "$-";
56     else
57         outputStream << '$';
58     outputStream << absDollars;
```

In the main function, cout is plugged in for outputStream.

For an alternate input algorithm, see Self-Test Exercise 3 in Chapter 7.

Overloaded >> Example:

Display 9.5 Overloading << and >> (4 of 5)

```
59     if (absCents >= 10)
60         outputStream << '.' << absCents;
61     else
62         outputStream << '.' << '0' << absCents;

63     return outputStream;
64 }
65
66 //Uses iostream and cstdlib:
67 istream& operator >>(istream& inputStream, Money& amount)
68 {
69     char dollarSign;
70     inputStream >> dollarSign; //hopefully
71     if (dollarSign != '$')
72     {
73         cout << "No dollar sign in Money input.\n";
74         exit(1);
75     }

76     double amountAsDouble;
77     inputStream >> amountAsDouble;
78     amount.dollars = amount.dollarsPart(amountAsDouble);
```

Returns a reference

In the main function, cin is plugged in for inputStream.

Since this is not a member operator, you need to specify a calling object for member functions of Money.

(continued)

Overloaded >> Example:

Display 8.5 Overloading << and >> (5 of 5)

Display 8.5 Overloading << and >>

```
79      amount.cents = amount.centsPart(amountAsDouble);  
80      return inputStream;  
81  }
```

Returns a reference

SAMPLE DIALOGUE

Enter an amount of money: \$123.45
Your amount is \$123.45
My amount is \$10.09.
One of us is richer.
\$123.45 + \$10.09 equals \$133.54
\$123.45 - \$10.09 equals \$113.36

Assignment Operator, =

- Must be overloaded as member operator
- Automatically overloaded
 - Default assignment operator:
 - Member-wise copy
 - Member variables from one object → corresponding member variables from other
- Default OK for simple classes
 - But with pointers → must write our own!

Increment and Decrement

- Each operator has two versions
 - Prefix notation: `++x`;
 - Postfix notation: `x++`;
- Must distinguish in overload
 - Standard overload method → Prefix
 - Add 2d parameter of type `int` → Postfix
 - Just a marker for compiler!
 - Specifies postfix is allowed

Overload Array Operator, []

- Can overload [] for your class
 - To be used with objects of your class
 - Operator must return a reference!
 - Operator [] must be a member function!

Inheritance

Introduction to Inheritance

- Object-oriented programming
 - Powerful programming technique
 - Provides abstraction dimension called *inheritance*
- General form of class is defined
 - Specialized versions then inherit properties of general class
 - And add to it/modify it's functionality for it's appropriate use

Inheritance Basics

- New class inherited from another class
- Base class
 - "General" class from which others derive
- Derived class
 - New class
 - Automatically has base class's:
 - Member variables
 - Member functions
 - Can then add additional member functions and variables

Derived Classes

- Consider example:
Class of "Employees"
- Composed of:
 - Salaried employees
 - Hourly employees
- Each is "subset" of employees
 - Another might be those paid fixed wage each month or week

Derived Classes

- Don't "need" type of generic "employee"
 - Since no one's just an "employee"
- General concept of employee helpful!
 - All have names
 - All have social security numbers
 - Associated functions for these "basics" are same among all employees
- So "general" class can contain all these "things" about employees

Employee Class

- Many members of "employee" class apply to all types of employees
 - Accessor functions
 - Mutator functions
 - Most data items:
 - SSN
 - Name
 - Pay
- We won't have "objects" of this class, however

Employee Class

- Consider printCheck() function:
 - Will always be "redefined" in derived classes
 - So different employee types can have different checks
 - Makes no sense really for "undifferentiated" employee
 - So function printCheck() in Employee class says just that
 - Error message stating "printCheck called for undifferentiated employee!! Aborting..."

Deriving from Employee Class

- Derived classes from Employee class:
 - Automatically have all member variables
 - Automatically have all member functions
- Derived class said to "inherit" members from base class
- Can then redefine existing members and/or add new members

```
class Employee
{
    //Access specifier
    public:
        //Declare a default constructor
        Employee( );
        //Declare the parameterized constructor
        Employee(string theName, string theSSN);
        //Declare the functions
        string getName( ) const;
        string getSSN( ) const;
        double getNetPay( ) const;
        void setName(string newName);
        void setSSN(string newSSN);
        void setNetPay(double newNetPay);
        void printCheck( ) const;
        //Access specifier
        private:
            //Declare required variables
            string name;
            string ssn;
            double netPay;
};
```


Display 14.3 Interface for the Derived Class HourlyEmployee (1 of 2)

Display 14.3 Interface for the Derived Class HourlyEmployee

```
1
2 //This is the header file hourlyemployee.h.
3 //This is the interface for the class HourlyEmployee.
4 #ifndef HOURLYEMPLOYEE_H
5 #define HOURLYEMPLOYEE_H

6 #include <string>
7 #include "employee.h"

8 using std::string;

9 namespace SavitchEmployees
10 {
```

Display 14.3 Interface for the Derived Class HourlyEmployee (1 of 2)

```
11  class HourlyEmployee : public Employee
12  {
13  public:
14      HourlyEmployee( );
15      HourlyEmployee(string theName, string theSsn,
16                      double theWageRate, double theHours);
17      void setRate(double newWageRate);
18      double getRate( ) const;
19      void setHours(double hoursWorked);
20      double getHours( ) const;
21      void printCheck( ) ;
22  private:
23      double wageRate;
24      double hours;
25  };

26  } //SavitchEmployees

27  #endif //HOURLYEMPLOYEE_H
```

You only list the declaration of an inherited member function if you want to change the definition of the function.

HourlyEmployee Class Interface

- Note definition begins same as any other
 - #ifndef structure
 - Includes required libraries
 - Also includes employee.h!
- And, the heading:

```
class HourlyEmployee : public Employee  
{ ...
```

 - Specifies "publicly inherited" from Employee class

HourlyEmployee Class Additions

- Derived class interface only lists new or "to be redefined" members
 - Since all others inherited are already defined
 - i.e.: "all" employees have ssn, name, etc.
- HourlyEmployee adds:
 - Constructors
 - wageRate, hours member variables
 - setRate(), getRate(), setHours(), getHours() member functions

HourlyEmployee Class Redefinitions

- HourlyEmployee redefines:
 - printCheck() member function
 - This "overrides" the printCheck() function implementation from Employee class
- It's definition must be in HourlyEmployee class's implementation
 - As do other member functions declared in HourlyEmployee's interface
 - New and "to be redefined"

Inheritance Terminology

- Common to simulate family relationships
- Parent class
 - Refers to base class
- Child class
 - Refers to derived class
- Ancestor class
 - Class that's a parent of a parent ...
- Descendant class
 - Opposite of ancestor

Constructors in Derived Classes

- Base class constructors are NOT inherited in derived classes!
 - But they can be invoked within derived class constructor
 - Which is all we need!
- Base class constructor must initialize all base class member variables
 - Those inherited by derived class
 - So derived class constructor simply calls it
 - "First" thing derived class constructor does

Derived Class Constructor Example

- Consider syntax for HourlyEmployee constructor:

```
HourlyEmployee::HourlyEmployee(string theName,  
                                string theNumber, double theWageRate,  
                                double theHours)  
    : Employee(theName, theNumber),  
      wageRate(theWageRate), hours(theHours)  
{  
    //Deliberately empty  
}
```

- Portion after : is "initialization section"
 - Includes invocation of Employee constructor

Another HourlyEmployee Constructor

- A second constructor:

```
HourlyEmployee::HourlyEmployee()  
    : Employee(), wageRate(o),  
      hours(o)  
{  
    //Deliberately empty  
}
```

- Default version of base class constructor is called (no arguments)
- Should always invoke one of the base class's constructors

Constructor: No Base Class Call

- Derived class constructor should always invoke one of the base class's constructors
- If you do not:
 - Default base class constructor automatically called
- Equivalent constructor definition:
HourlyEmployee::HourlyEmployee()
 : wageRate(o), hours(o)

{ }

Pitfall: Base Class Private Data

- Derived class "inherits" private member variables
 - But still cannot directly access them
 - Not even through derived class member functions!
- Private member variables can ONLY be accessed "by name" in member functions of the class they're defined in

Pitfall: Base Class Private Member Functions

- Same holds for base class member functions
 - Cannot be accessed outside interface and implementation of base class
 - Not even in derived class member function definitions

Pitfall: Base Class Private Member Functions Impact

- Larger impact here vs. member variables
 - Member variables can be accessed indirectly via accessor or mutator member functions
 - Member functions simply not available
- This is "reasonable"
 - Private member functions should be simply "helper" functions
 - Should be used only in class they're defined

The protected: Qualifier

- New classification of class members
- Allows access "by name" in derived class
 - But nowhere else
 - Still no access "by name" in other classes
- In class it's defined → acts like private
- Considered "protected" in derived class
 - To allow future derivations
- Many feel this "violates" information hiding

Redefinition of Member Functions

- Recall interface of derived class:
 - Contains declarations for new member functions
 - Also contains declarations for inherited member functions to be changed
 - Inherited member functions NOT declared:
 - Automatically inherited unchanged
- Implementation of derived class will:
 - Define new member functions
 - Redefine inherited functions as declared

Redefining vs. Overloading

- Very different!
- Redefining in derived class:
 - SAME parameter list
 - Essentially "re-writes" same function
- Overloading:
 - Different parameter list
 - Defined "new" function that takes different parameters
 - Overloaded functions must have different signatures

Accessing Redefined Base Function

- When redefined in derived class, base class's definition not "lost"
- Can specify it's use:
Employee JaneE;
HourlyEmployee SallyH;
JaneE.printCheck(); → calls Employee's
printCheck function
SallyH.printCheck(); → calls HourlyEmployee
printCheck function
SallyH.Employee::printCheck(); → Calls Employee's
printCheck function!
- Not typical here, but useful sometimes

Functions Not Inherited

- All "normal" functions in base class are inherited in derived class
- Exceptions:
 - Constructors (we've seen)
 - Destructors
 - Copy constructor
 - But if not defined, generates "default" one
 - Recall need to define one for pointers!
 - Assignment operator
 - If not defined → default

Destructor

```
class Demo {  
public:  
    Demo(int pa) {  
        std::cout << "constructor called.." << std::endl;  
        aPtr = new int;  
        *aPtr = pa;  
    }  
    ~Demo() {  
        std::cout << "destructor called.." << std::endl;  
        delete aPtr;  
    }  
    void do_something() {  
        std::cout << *aPtr << std::endl;  
    }  
private:  
    int *aPtr;  
};
```

Copy Constructor

```
class Demo {  
public:  
    Demo(Demo &myDemo) {  
        std::cout << "copy constructor called.." << std::endl;  
        aPtr = new int;  
        *aPtr = *(myDemo.aPtr);  
    }  
    ...  
private:  
    int *aPtr;  
};
```

何時需要？

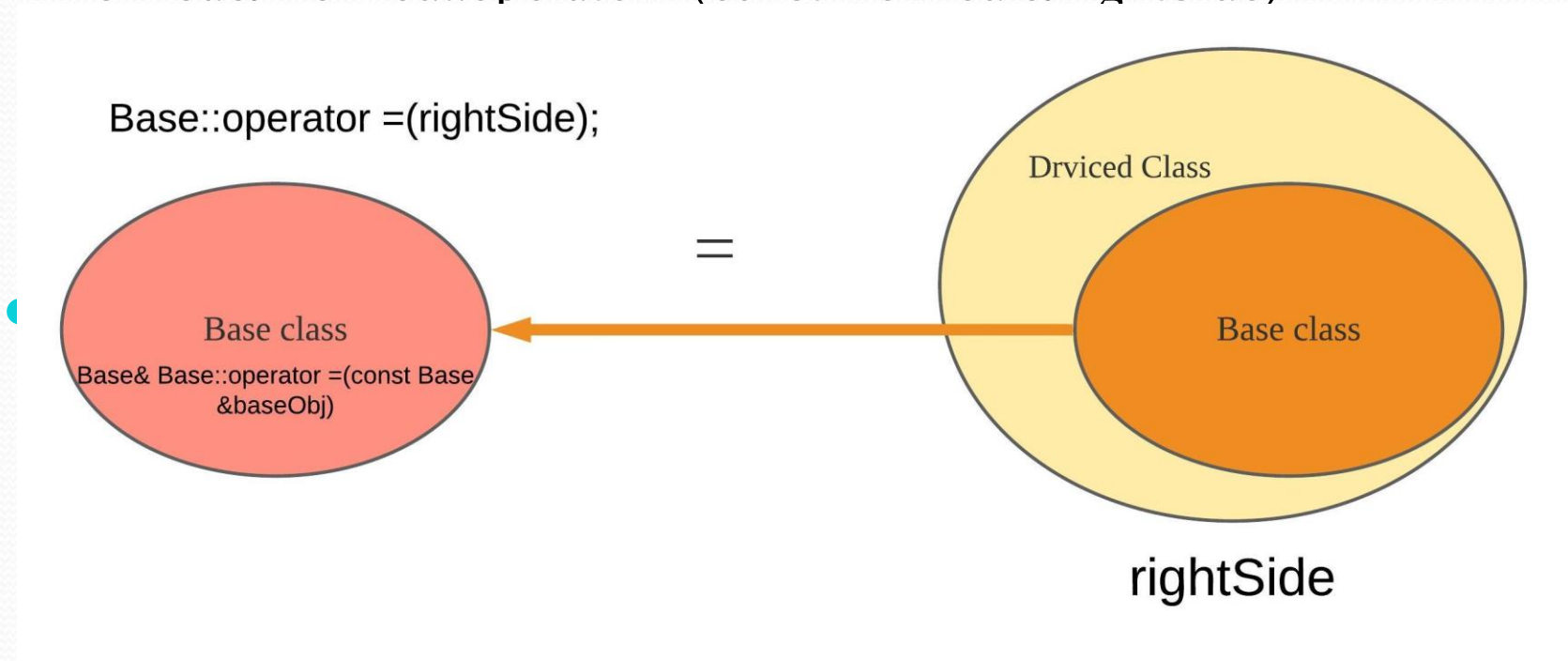
當類別中有資料型別為指標時。因為預設的複雜建構子進行進行物件的成員複製時，僅會複製指標型態屬性位址值，而不是指標所指向的值。因此會造成兩個物件的指標會指向同一個內容。當類別沒有指標，而所複製物件的動作只需要把物件的所有屬性複製一次，您並不需要自行複製建構子，直接利用系統自動產生（內定）的複製建構子即可。

Assignment Operators and Copy Constructors

- Recall: overloaded assignment operators and copy constructors
NOT inherited
 - But can be used in derived class definitions
 - Typically MUST be used!
 - Similar to how derived class constructor invokes base class constructor

Assignment Operator Example

- Given "Derived" is derived from "Base":
Derived& Derived::operator =(const Derived & rightSide)




```
class BaseClass {
public:
    // Base class assignment operator
    BaseClass& operator=(const BaseClass& other) {
        // Perform assignment for the base class
        return *this;
    }
};

class DerivedClass : public BaseClass {
public:
    // Derived class assignment operator
    DerivedClass& operator=(const DerivedClass& other) {
        // Call base class assignment operator
        BaseClass::operator=(other);

        // Perform assignment for the derived class
        return *this;
    }
};
```

Copy Constructor Example

- Consider:

```
Derived::Derived(const Derived& Object)
                : Base(Object), ...
{...}
```

- After : is invocation of base copy constructor
 - Sets inherited member variables of derived class object being created
 - Note Object is of type Derived; but it's also of type Base, so argument is valid

Destructors in Derived Classes

- If base class destructor functions correctly
 - Easy to write derived class destructor
- When derived class destructor is invoked:
 - Automatically calls base class destructor!
 - So no need for explicit call
- So derived class destructors need only be concerned with derived class variables
 - And any data they "point" to
 - Base class destructor handles inherited data automatically

Recall: Copy Constructor!

- 如果兩個物件的 `private member data` 指向同一個記憶體空間。
- **Destructor** 就會刪除了可能你不想刪除的東西了！
- 物件的資料不要可以跟別人共用。（又不是連體嬰）
 - 除了 `static member data`

Destructor Calling Order

- Consider:
class B derives from class A
class C derives from class B
 $A \leftarrow B \leftarrow C$
- When object of class C goes out of scope:
 - Class C destructor called 1st
 - Then class B destructor called
 - Finally class A destructor is called
- Opposite of how constructors are called

"Is a" vs. "Has a" Relationships

- Inheritance
 - Considered an "Is a" class relationship
 - e.g., An HourlyEmployee "is a" Employee
 - A Convertible "is an" Automobile
- A class contains objects of another class as it's member data
 - Considered a "Has a" class relationship
 - e.g., One class "has a" object of another class as it's data

Protected and Private Inheritance

- New inheritance "forms"
 - Both are rarely used
- Protected inheritance:
class SalariedEmployee : protected Employee
{...}
 - Public members in base class become protected in derived class
- Private inheritance:
class SalariedEmployee : private Employee
{...}
 - All members in base class become private in derived class

Multiple Inheritance

- Derived class can have more than one base class!
 - Syntax just includes all base classes separated by commas:
class derivedMulti : public base1, base2
{...}
- Possibilities for ambiguity are endless!
- Dangerous undertaking!
 - Some believe should never be used
 - Certainly should only be used by experienced programmers!